

<

Cours Modélisation et vérification des systèmes informatiques

Exercices

Modélisation d'algorithmes en PlusCal (I)

par Dominique Méry

7 octobre 2026

Exercice 1 *appex4...*

Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA contenant une expression de l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence d'erreurs à l'exécution.

Question 1.1 *appex4_1_1.tla*

$$\begin{aligned} \ell_1 : x = 10 \wedge y = z+x \wedge z = 2 \cdot x \\ y := z+x \\ \ell_2 : x = 10 \wedge y = x+2 \cdot 10 \end{aligned}$$
◇ **Solution de l'exercice 1**

Listing 1 – Module TLA+

```

1  ----- MODULE appex4_1_1 -----
2  EXTENDS Integers, TLC
3  -----
4  CONSTANTS x0, y0, z0
5  -----
6  pre(a, b, c) == a=10 /\ c=2*a /\ b=c+a
7  -----
8  (* Verifying the existence of a triple that satisfies pre(x0,y0,z0) *)
9  ASSUME pre(x0, y0, z0)
10 -----
11 (*
12 --algorithm test {
13 variables x=x0, z=z0, y=y0, t;
14 {
15 l1: \* assert x=10 /\ z=2*x /\ y = z + x;
16 y:=z+x;
17 l2: \* assert x = 10 /\ y = x + 2*10
18 skip ;
19 }
20 }
21 *)
22 \* BEGIN TRANSLATION (chksum(pcal) = "cec661f" /\ chksum(tla) = "51e99e7b")
23 CONSTANT defaultInitValue
24 VARIABLES x, z, y, t, pc
25
26 vars == << x, z, y, t, pc >>
27
28 Init == (* Global variables *)
29         /\ x = x0
30         /\ z = z0
31         /\ y = y0
32         /\ t = defaultInitValue
33         /\ pc = "l1"
34
35 l1 == /\ pc = "l1"
36        /\ y' = z+x
37        /\ pc' = "l2"

```

```

38         /\ UNCHANGED << x, z, t >>
39
40 l2 == /\ pc = "l2"
41        /\ TRUE
42        /\ pc' = "Done"
43        /\ UNCHANGED << x, z, y, t >>
44
45 (* Allow infinite stuttering to prevent deadlock on termination. *)
46 Terminating == pc = "Done" /\ UNCHANGED vars
47
48 Next == l1 \/ l2
49        \/ Terminating
50
51 Spec == Init /\ [][Next]_vars
52
53 Termination == <>(pc = "Done")
54
55 \* END TRANSLATION
56
57 \* ASSUME pre
58 -----
59 (* preliminaries for checking on possible values *)
60 MAX == 32767 (* 16 bits *)
61 D == -32768..32767
62 DD(X) == (X # defaultInitValue) => (X \in D)
63 -----
64 Inv ==
65     /\ pc \in {"l1", "l2", "Done"}
66     /\ x \in Int /\ y \in Int /\ z \in Int
67     /\ pc="l1" => x=10 /\ z=2*x /\ y=z+x
68     /\ pc="l2" => x=10 /\ y=x+2*10
69     /\ pc="Done" => x=10 /\ y=x+2*10
70 Safety_Partial_Correctness == pc="Done" => x=10 /\ y=x+2*10
71 Safety_rte == DD(x) /\ DD(y) /\ DD(z) /\ DD(t)
72 check == Inv /\ Safety_Partial_Correctness /\ Safety_rte
73 prop == [] (x=x0)
74 =====

```

Listing 2 – Configuration TLC

```

1 SPECIFICATION Spec
2 CONSTANT defaultInitValue = defaultInitValue
3 \* Add statements after this line.
4 \* SPECIFICATION
5 \* Uncomment the previous line and provide the specification name if it's
   declared
6 \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
7
8 CONSTANTS
9     greeting = "Hello"
10    x0=10
11    y0 =30
12    z0 = 20
13    defaultInitValue = 50000
14 INIT Init
15 NEXT Next
16
17 \* PROPERTY
18 \* Uncomment the previous line and add property names
19

```

```

20 INVARIANT
21 \* Uncomment the previous line and add invariant names
22 Inv

```

Fin 1

Question 1.2 On suppose que p est un nombre premier :

$$\begin{aligned} \ell_1 : x &= 2^p \wedge y = 2^{p+1} \wedge x \cdot y = 2^{2 \cdot p+1} \\ x &:= y + x + 2^x \\ \ell_2 : x &= 5 \cdot 2^p \wedge y = 2^{p+1} \end{aligned}$$

◊ **Solution de l'exercice 1**

Listing 3 – Module TLA+

```

1  ----- MODULE appex4_1_2 -----
2  EXTENDS Integers, Naturals, TLC
3  CONSTANTS x0, y0, p, maxi
4  -----
5  diviseurs(n) == { m \in 1..maxi: n % m = 0 }
6  premier(n) == diviseurs(n) = {1, n} /\ n # 1
7  pre == x0=2^p /\ y0=2^(p+1) /\ x0*y0=2^(2*p+1)
8  -----
9  ASSUME premier(p) /\ pre
10 (*
11
12 --algorithm test {
13 variables x=x0, y=y0;
14 {
15 l1:assert x=2^p /\ y=(2^p)*2 /\ x*y=2^(2*p+1);
16 x:=y+x+(2^x);
17 l2:assert x=5*(2^p) /\ y=2^(p+1) ;
18 }
19 }
20 *)
21 \* BEGIN TRANSLATION (chksum(pcal) = "31a0954b" /\ chksum(tla) = "e1037380")
22 VARIABLES x, y, pc
23
24 vars == << x, y, pc >>
25
26 Init == (* Global variables *)
27         /\ x = x0
28         /\ y = y0
29         /\ pc = "l1"
30
31 l1 == /\ pc = "l1"
32        /\ Assert (x=2^p /\ y=(2^p)*2 /\ x*y=2^(2*p+1),
33                  "Failure_of_assertion_at_line_16,_column_4.")
34        /\ x' = y+x+(2^x)
35        /\ pc' = "l2"
36        /\ y' = y
37
38 l2 == /\ pc = "l2"
39        /\ Assert (x=5*(2^p) /\ y=2^(p+1),
40                  "Failure_of_assertion_at_line_18,_column_4.")
41        /\ pc' = "Done"
42        /\ UNCHANGED << x, y >>
43
44 (* Allow infinite stuttering to prevent deadlock on termination. *)
45 Terminating == pc = "Done" /\ UNCHANGED vars
46

```

```

47 Next == l1 \/ l2
48         \/ Terminating
49
50 Spec == Init /\ [][Next]_vars
51
52 Termination == <>(pc = "Done")
53
54 \* END TRANSLATION
55
56 -----
57 MAX == 32768 (* 16 bits *)
58 D == 0..32768
59 DD(X) == ( X \in D)
60 DDP(X) == (X # defaultInitValue) => (X \in D)
61 Safety_absence == DD(x) /\ DD(y)
62 Inv ==
63     /\ pc \in {"l1", "l2", "Done"}
64     /\ x \in Int /\ y \in Int
65     /\ pc="l1" => x=2^p /\ y=2^p*2 /\ x*y=2^(2*p+1)
66     /\ pc="l2" => x=5*2^p /\ y=2^(p+1)
67
68 =====

```

Listing 4 – Configuration TLC

```

1 \* SPECIFICATION
2 \* Uncomment the previous line and provide the specification name if it's
   declared
3 \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5 CONSTANTS
6     greeting = "Hello"
7     p = 2
8     x0 = 4
9     y0 = 8
10    maxi = 89
11
12 INIT Init
13 NEXT Next
14
15 \* PROPERTY
16 \* Uncomment the previous line and add property names
17
18 \* INVARIANT
19 \* Uncomment the previous line and add invariant names

```

Fin 1

Question 1.3

$$\begin{aligned}
 \ell_1 : x = 1 \wedge y = 12 \\
 x := 2 \cdot y \\
 \ell_2 : x = 1 \wedge y = 24
 \end{aligned}$$

◇ Solution de l'exercice 1

Listing 5 – Module TLA+

```

1 ----- MODULE appex4_1_3 -----
2 EXTENDS Integers, TLC
3 -----

```

```

4  CONSTANTS x0,y0
5  -----
6  pre == x0=1 /\ y0=12
7
8  -----
9  (*
10
11  --algorithm test {
12  variables x=x0,y=y0;
13  {
14  l1:assert x=1 /\ y=12;
15  x:=2*y;
16  l2:assert x=24 /\ y=12;
17  }
18  }
19  *)
20  \* BEGIN TRANSLATION (chksum(pcal) = "fdb997c5" /\ chksum(tla) = "97306b1b")
21  VARIABLES x, y, pc
22
23  vars == << x, y, pc >>
24
25  Init == (* Global variables *)
26          /\ x = x0
27          /\ y = y0
28          /\ pc = "l1"
29
30  l1 == /\ pc = "l1"
31          /\ Assert(x=1 /\ y=12, "Failure_of_assertion_at_line_14,_column_4.")
32          /\ x' = 2*y
33          /\ pc' = "l2"
34          /\ y' = y
35
36  l2 == /\ pc = "l2"
37          /\ Assert(x=24 /\ y=12, "Failure_of_assertion_at_line_16,_column_4.")
38          /\ pc' = "Done"
39          /\ UNCHANGED << x, y >>
40
41  (* Allow infinite stuttering to prevent deadlock on termination. *)
42  Terminating == pc = "Done" /\ UNCHANGED vars
43
44  Next == l1 \/ l2
45          \/ Terminating
46
47  Spec == Init /\ [][Next]_vars
48
49  Termination == <>(pc = "Done")
50
51  \* END TRANSLATION
52  -----
53
54  MAX == 32768 (* 16 bits *)
55  D == 0..32768
56  DD(X) == ( X \in D)
57  DDP(X) == (X # defaultInitValue) => (X \in D)
58  -----
59  Safety_absence == DD(x) /\ DD(y)
60  Inv ==
61      /\ pc \in {"l1", "l2", "Done"}
62      /\ x \in Int /\ y \in Int
63      /\ pc="l1" => x=1 /\ y=12
64      /\ pc="l2" => x=1 /\ y=24

```

```

65
66 =====

```

Listing 6 – Configuration TLC

```

1  \* SPECIFICATION
2  \* Uncomment the previous line and provide the specification name if it's
   declared
3  \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5  CONSTANTS
6      greeting = "Hello"
7
8  INIT Init
9  NEXT Next
10
11 \* PROPERTY
12 \* Uncomment the previous line and add property names
13
14 \* INVARIANT
15 \* Uncomment the previous line and add invariant names

```

Fin 1

Question 1.4

$$\begin{aligned} \ell_1 : x = 11 \wedge y = 13 \\ z := x; x := y; y := z; \\ \ell_2 : x = 26/2 \wedge y = 33/3 \end{aligned}$$

◇– Solution de l'exercice 1

Listing 7 – Module TLA+

```

1  ----- MODULE appex4_1_4 -----
2  EXTENDS Integers, TLC
3  -----
4
5  (*
6  --algorithm test {
7  variables x=11,y=13,z;
8  {
9  l1:assert x=11 /\ y=13;
10 z:=x;x:=y;y:=z;
11 l2:assert x=26 \div 2 /\ y=33 \div 3;
12
13 l3:print <<x,y>>;
14 }
15 }
16 *)
17 \* BEGIN TRANSLATION
18 CONSTANT defaultInitValue
19 VARIABLES x, y, z, pc
20
21 vars == << x, y, z, pc >>
22
23 Init == (* Global variables *)
24         /\ x = 11
25         /\ y = 13
26         /\ z = defaultInitValue
27         /\ pc = "l1"

```

```

28
29 l1 == /\ pc = "l1"
30      /\ Assert(x=11 /\ y=13, "Failure_of_assertion_at_line_10,_column_4.")
31      /\ z' = x
32      /\ x' = y
33      /\ y' = z'
34      /\ pc' = "l2"
35
36 l2 == /\ pc = "l2"
37      /\ Assert(x=26 \div 2 /\ y=33 \div 3,
38              "Failure_of_assertion_at_line_12,_column_4.")
39      /\ pc' = "l3"
40      /\ UNCHANGED << x, y, z >>
41
42 l3 == /\ pc = "l3"
43      /\ PrintT(<<x,y>>)
44      /\ pc' = "Done"
45      /\ UNCHANGED << x, y, z >>
46
47 (* Allow infinite stuttering to prevent deadlock on termination. *)
48 Terminating == pc = "Done" /\ UNCHANGED vars
49
50 Next == l1 \/ l2 \/ l3
51        \/ Terminating
52
53 Spec == Init /\ [][Next]_vars
54
55 Termination == <>(pc = "Done")
56
57 \* END TRANSLATION
58
59 -----
60 MAX == 32768 (* 16 bits *)
61 D == 0..32768
62 DD(X) == ( X \in D)
63 DDP(X) == (X # defaultInitValue) => (X \in D)
64 -----
65 Safety_absence == DD(x) /\ DD(y)
66 Inv ==
67     /\ pc \in {"l1", "l2", "Done"}
68     /\ x \in Int /\ y \in Int
69     /\ pc="l1" => x=11 /\ y=13
70     /\ pc="l2" => x=26 \div 2 /\ y=33 \div 3
71
72 =====

```

Listing 8 – Configuration TLC

```

1 \* SPECIFICATION
2 \* Uncomment the previous line and provide the specification name if it's
   declared
3 \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5 CONSTANTS
6     greeting = "Hello"
7
8 INIT Init
9 NEXT Next
10
11 \* PROPERTY

```

```

12 \* Uncomment the previous line and add property names
13
14 \* INVARIANT
15 \* Uncomment the previous line and add invariant names

```

Fin 1

Question 1.5 Dans cette question, on considère l'algorithme correspondant au calcul du maximum de deux nombres.

Requires : $x_0, y_0 \in \mathbb{N}$
Ensurees : $z_f = \max(x_0, y_0)$

```

 $\ell_0 : \{\dots\}$ 
if  $X < Y$  then
   $\ell_1 : \{\dots\}$ 
   $Z := Y;$ 
   $\ell_2 : \{\dots\}$ 
else
   $\ell_3 : \{\dots\}$ 
   $Z := X;$ 
   $\ell_4 : \{\dots\}$ 
;
 $\ell_5 : \{\dots\}$ 

```

Algorithme 1: MAX annotée

◇ Solution de l'exercice 1

Listing 9 – Module TLA+

```

1 ----- MODULE appex4_1_5 -----
2 EXTENDS Naturals, Integers, TLC
3
4 CONSTANTS x0,y0,min,max
5
6 ASSUME x0 \in min..max /\ y0 \in min..max
7
8
9 (*
10 --algorithm commeonveut {
11   variables x=x0,
12             y = y0,
13             z;
14
15 {
16   10:assert x=x0 /\ y=y0;
17   if ( x < y ) {
18     11: assert x=x0 /\ y=y0 /\ x<y;
19     z:=y;
20     12: assert x=x0 /\ y=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y} ;
21   }
22 else
23 {
24   13: assert x=x0 /\ y=y0 /\ x >= y ;
25   z:= x;

```

```

26 | 14: assert x=x0 /\ y=y0 /\ z=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y};
27 | };
28 | 15: assert x=x0 /\ y=y0 /\ z \in {x,y} /\ x <= z /\ y <= z;
29 | print "done";
30 | 16: assert x=x0 /\ y=y0 /\ z \in {x,y} /\ x <= z /\ y <= z;
31 |
32 | }
33 | }
34 | *)
35 | \* BEGIN TRANSLATION
36 | CONSTANT defaultInitValue
37 | VARIABLES x, y, z, pc
38 |
39 | vars == << x, y, z, pc >>
40 |
41 | Init == (* Global variables *)
42 |         /\ x = x0
43 |         /\ y = y0
44 |         /\ z = defaultInitValue
45 |         /\ pc = "10"
46 |
47 | 10 == /\ pc = "10"
48 |        /\ Assert(x=x0 /\ y=y0, "Failure_of_assertion_at_line_16,_column_5.")
49 |        /\ IF x < y
50 |            THEN /\ pc' = "11"
51 |            ELSE /\ pc' = "13"
52 |        /\ UNCHANGED << x, y, z >>
53 |
54 | 11 == /\ pc = "11"
55 |        /\ Assert(x=x0 /\ y=y0 /\ x<y,
56 |            "Failure_of_assertion_at_line_18,_column_8.")
57 |        /\ z' = y
58 |        /\ pc' = "12"
59 |        /\ UNCHANGED << x, y >>
60 |
61 | 12 == /\ pc = "12"
62 |        /\ Assert(x=x0 /\ y=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y},
63 |            "Failure_of_assertion_at_line_20,_column_7.")
64 |        /\ pc' = "15"
65 |        /\ UNCHANGED << x, y, z >>
66 |
67 | 13 == /\ pc = "13"
68 |        /\ Assert(x=x0 /\ y=y0 /\ x >= y,
69 |            "Failure_of_assertion_at_line_24,_column_6.")
70 |        /\ z' = x
71 |        /\ pc' = "14"
72 |        /\ UNCHANGED << x, y >>
73 |
74 | 14 == /\ pc = "14"
75 |        /\ Assert(x=x0 /\ y=y0 /\ z=y0 /\ x \leq z /\ y \leq z /\ z \in {x,y},
76 |            "Failure_of_assertion_at_line_26,_column_6.")
77 |        /\ pc' = "15"
78 |        /\ UNCHANGED << x, y, z >>
79 |
80 | 15 == /\ pc = "15"
81 |        /\ Assert(x=x0 /\ y=y0 /\ z \in {x,y} /\ x <= z /\ y <= z,
82 |            "Failure_of_assertion_at_line_28,_column_6.")
83 |        /\ PrintT("done")
84 |        /\ pc' = "16"
85 |        /\ UNCHANGED << x, y, z >>
86 |

```

```

87 l6 == /\ pc = "l6"
88     /\ Assert(x=x0 /\ y=y0 /\ z \in {x,y} /\ x <= z /\ y <= z,
89           "Failure_of_assertion_at_line_30,_column_6.")
90     /\ pc' = "Done"
91     /\ UNCHANGED << x, y, z >>
92
93 (* Allow infinite stuttering to prevent deadlock on termination. *)
94 Terminating == pc = "Done" /\ UNCHANGED vars
95
96 Next == l0 \/ l1 \/ l2 \/ l3 \/ l4 \/ l5 \/ l6
97        \/ Terminating
98
99 Spec == Init /\ [][Next]_vars
100
101 Termination == <>(pc = "Done")
102
103 \* END TRANSLATION
104
105
106 maximum(X,Y,Z) == Z \in {X,Y} /\ X <= Z /\ Y <= Z
107
108
109 Inv ==
110     /\ pc \in {"l0","l1","l2","l3","l4","l5","l6","Done"}
111     /\ x \in Int /\ y \in Int /\ (z#defaultInitValue => z \in Int)
112     /\ pc = "l0" => x=x0 /\ y=y0
113     /\ pc="l1" => x=x0 /\ y=y0 /\ x<y
114
115 safetyrte == x \in min..max /\ y \in min..max /\ z \in min..max
116
117
118 safetypcv1 == pc="Done" => x=x0 /\ y=y0 /\ z \in {x,y} /\ x <= z /\ y <= z
119
120
121 safetypcv2 == pc="Done" => x=x0 /\ y=y0 /\ maximum(x,y,z)
122
123 =====
124 \* Modification History
125 \* Last modified Tue Oct 04 09:20:27 CEST 2022 by mery
126 \* Created Wed Sep 09 17:02:47 CEST 2015 by mery

```

Listing 10 – Configuration TLC

```

1 \* SPECIFICATION
2 \* Uncomment the previous line and provide the specification name if it's
   declared
3 \* in the specification file. Comment INIT / NEXT parameters if you use
   SPECIFICATION.
4
5 CONSTANTS
6     greeting = "Hello"
7
8     x0 = 89
9     y0 = 789
10    min = 0
11    max = 888888
12    defaultInitValue = 99999999
13
14    INIT Init
15    NEXT Next
16

```

```

17 \* PROPERTY
18 \* Uncomment the previous line and add property names
19
20 \* INVARIANT
21 \* Uncomment the previous line and add invariant names

```

Fin 1

Exercice 2 ✓

On considère l'algorithme suivant :

```

2)  START
    { $x_1 \geq 0 \wedge x_2 > 0$ }
     $(y_1, y_2, y_3) \leftarrow (x_1, 0, x_2)$ ;
    while  $y_3 \leq y_1$  do  $y_3 \leftarrow 2y_3$ ;
    while  $y_3 \neq x_2$  do
        begin  $(y_2, y_3) \leftarrow (2y_2, y_3/2)$ ;
        if  $y_3 \leq y_1$  do  $(y_1, y_2) \leftarrow (y_1 - y_3, y_2 + 1)$ 
        end;
     $(z_1, z_2) \leftarrow (y_1, y_2)$ 
    { $0 \leq z_1 < x_2 \wedge x_1 = z_2x_2 + z_1$ }
    HALT

```

Question 2.1 Montrer que cet algorithme est aprtiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du langage PlusCal.

Question 2.2 Montrer qu'il est sans erreur à l'exécution.

Exercice 3 ✓

On considère l'algorithme suivant :

START

$\{x_1 > 0 \wedge x_2 > 0\}$

$(y_1, y_2, y_3, y_4) \leftarrow (x_1, x_2, x_2, 0);$

while $y_1 \neq y_2$ **do**

begin while $y_1 > y_2$ **do** $(y_1, y_4) \leftarrow (y_1 - y_2, y_4 + y_3);$

end; while $y_2 > y_1$ **do** $(y_2, y_3) \leftarrow (y_2 - y_1, y_3 + y_4)$

$(z_1, z_2) \leftarrow (y_1, y_3 + y_4)$

$\{z_1 = \text{gcd}(x_1, x_2) \wedge z_2 = \text{scm}(x_1, x_2)\}$

HALT

where $\text{scm}(x_1, x_2)$ is the smallest common multiple of x_1 and x_2 ; that is, it is their product divided by the greatest common divisor.

Question 3.1 Montrer que cet algorithme est aptriellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du langage PlusCal.

Question 3.2 Montrer qu'il est sans erreur à l'exécution.

◇ **Solution de l'exercice 3**

Fin 3

Exercice 4 ✓

Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA contenant une expression de l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence d'erreurs à l'exécution.

Question 4.1 Soit l'annotation suivante :

$\ell_1 : x = 10 \wedge y = z + x \wedge z = 2 \cdot x$

$y := z + x$

$\ell_2 : x = 10 \wedge y = x + 2 \cdot 10$

Traduire en PlusCal.

Exercice 5 On considère l'algorithme suivant :

```

START
{ $x_1 > 0 \wedge x_2 > 0$ }
( $y_1, y_2, y_3, y_4$ )  $\leftarrow$  ( $x_1, x_2, x_2, 0$ );
while  $y_1 \neq y_2$  do
  begin while  $y_1 > y_2$  do ( $y_1, y_4$ )  $\leftarrow$  ( $y_1 - y_2, y_4 + y_3$ );
  end; while  $y_2 > y_1$  do ( $y_2, y_3$ )  $\leftarrow$  ( $y_2 - y_1, y_3 + y_4$ );
end;
( $z_1, z_2$ )  $\leftarrow$  ( $y_1, y_3 + y_4$ )
{ $z_1 = \gcd(x_1, x_2) \wedge z_2 = \text{scm}(x_1, x_2)$ }
HALT

```

where $\text{scm}(x_1, x_2)$ is the smallest common multiple of x_1 and x_2 ; that is, it is their product divided by the greatest common divisor.

Question 5.1 Montrer que cet algorithme est aptriellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du lanage PlusCal.

Question 5.2 Montrer qu'il est sans erreur à l'exécution.

Exercice 6 Exponentiation en PlusCal

Ecrire l'algorithme de l'exponentiation avec PlusCal.

On suppose que x_1 et x_2 sont des constantes.

```

precondition :  $x_1 \in \mathbb{N} \wedge x_2 \in \mathbb{N} \wedge x_1 \neq 0$ 
postcondition :  $z = x_1^{x_2}$ 
local variables :  $y_1, y_2, y_3 \in \mathbb{Z}$ 

 $\ell_0 : \{y_1, y_2, y_3, z \in \mathbb{Z}\}$ 
( $y_1, y_2, y_3$ ) := ( $x_1, x_2, 1$ );
 $\ell_1 : \{y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
while  $y_2 \neq 0$  do
   $\ell_2 : \{y_2 \neq 0 \wedge y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
  if impair( $y_2$ ) then
     $\ell_3 : \{\text{impair}(y_2) \wedge y_2 \neq 0 \wedge \dots \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
     $y_2 := y_2 - 1$ ;
     $\ell_4 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge y_3 \cdot y_1 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
     $y_3 := y_3 \cdot y_1$ ;
     $\ell_5 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
  end if
  ;
   $\ell_6 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge \dots \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
   $y_1 := y_1 \cdot y_1$ ;
   $\ell_7 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge y_3 \cdot y_1^{y_2 \text{ div } 2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
   $y_2 := y_2 \text{ div } 2$ ;
   $\ell_8 : \{y_2 \geq 0 \wedge \dots \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
;
 $\ell_9 : \{y_2 = 0 \wedge \dots \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
 $z := y_3$ ;
 $\ell_{10} : \{y_2 = 0 \wedge y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z} \wedge z = x_1^{x_2}\}$ 

```

Algorithme 2: Algorithme de l'exponentiation indienne annoté

Exercice 7 Vérifier le programme C suivant en la traduisant dans un module PlusCal. Pour cela, on conservera les éléments algorithmiques suffisants pour analyser le programme C. Par exemple, on ne traduira pas les instructions de lecture ou les inclusions de bibliothèques.

```

1
2
3  /* C Program to find roots of a quadratic equation when coefficients are
   entered by user. */
4  /* Library function sqrt() computes the square root. */
5
6  #include <stdio.h>
7  #include <math.h> /* This is needed to use sqrt() function.*/
8  int main()
9  {
10     float a, b, c, determinant, r1,r2, real, imag;
11     printf("Enter coefficients a, b and c: ");
12     scanf("%f%f%f", &a, &b, &c);
13     determinant=b*b-4*a*c;
14     if (determinant>0)
15     {
16         r1= (-b+sqrt(determinant))/(2*a);
17         r2= (-b-sqrt(determinant))/(2*a);
18         printf("Roots are: %.2f and %.2f", r1 , r2);
19     }
20     else if (determinant==0)
21     {
22         r1 = r2 = -b/(2*a);
23         printf("Roots are: %.2f and %.2f", r1, r2);
24     }
25     else
26     {
27         real= -b/(2*a);
28         imag = sqrt(-determinant)/(2*a);
29         printf("Roots are: %.2f+%.2fi and %.2f-%.2fi", real, imag, real, imag);
30     }
31     return 0;
32 }
33
34 Output 1
35
36
37 Enter coefficients a, b and c: 2.3
38 4
39 5.6
40 Roots are: -0.87+1.30i and -0.87-1.30i
41
42 Output 2
43
44
45 Enter coefficients a, b and c: 4
46 1
47 0
48 Roots are: 0.00 and -0.25
49
50 To solve this program, library function sqrt() is used. This function
   calculates the square root of a number. Learn more about, sqrt() function.
51 Similar C Programming Examples
52 C Program to Add Two Complex Numbers by Passing Structure to a Function
53 C Program to Find Quotient and Remainder of Two Integers Entered by User
54 C Program to Check Prime or Armstrong Number Using User-defined Function
55 C Program to Display Prime Numbers Between Intervals Using User-defined

```

Function
56 C Program to Display its own Source Code as Output

Exercice 8

Vérifier le programme C suivant en la traduisant dans un module PlusCal.

```
1  /* C program to check whether a number is palindrome or not */
2
3  #include <stdio.h>
4  int main()
5  {
6      int n, reverse=0, rem,temp;
7      printf("Enter_an_integer:_");
8      scanf("%d", &n);
9      temp=n;
10     while(temp!=0)
11     {
12         rem=temp%10;
13         reverse=reverse*10+rem;
14         temp/=10;
15     }
16     /* Checking if number entered by user and it's reverse number is equal. */
17     if(reverse==n)
18         printf("%d_is_a_palindrome.",n);
19     else
20         printf("%d_is_not_a_palindrome.",n);
21     return 0;
22 }
```

◇ Solution de l'exercice 8

Fin 8

Exercice 9 Vérifier le programme C suivant en la traduisant dans un module PlusCal.

```
1  #include <stdio.h>
2  int main()
3  {
4      int n,count=0;
5      printf("Enter_an_integer:_");
6      scanf("%d", &n);
7      while(n!=0)
8      {
9          n/=10;          /* n=n/10 */
10         ++count;
11     }
12     printf("Number_of_digits:_%d",count);
13 }
```

Exercice 10 Nous avons récupéré un programme C qui réalise le tri à bulles. Traduire ce programme en PlusCal et analyser sa correction partielle et l'absence d'erreurs à l'exécution.

```
1
2
3  void tri_a_bulle(int* t, int const size)
4  {
5      int en_desordre = 1;
6      int i,j;
7
8      for (i = 0; (i < size) && en_desordre; ++i)
```

```

9      {
10          en_desordre = 0;
11          for (j = 1; j < (size - i); ++j)
12          {
13              if (t[j-1] > t[j])
14              {
15                  int temp = t[j-1];
16                  t[j-1] = t[j];
17                  t[j] = temp;
18                  en_desordre = 1;
19              }
20          }
21      }
22  }

```

```

1      #include<algorithm> // pour la fonction swap
2
3      void tri_a_bulle(int *t, int const size)
4      {
5          bool en_desordre = true;
6          for (int i = 0; i < size && en_desordre; ++i)
7          {
8              en_desordre = false;
9              for (int j = 1; j < size - i; ++j)
10                 if (t[j-1] > t[j])
11                 {
12                     std::swap(t[j], t[j-1]);
13                     en_desordre = true;
14                 }
15          }
16      }

```

Exercice 11 Analyser le programme C suivant qui réalise le tri par sélection.

```

1      void selection(int *t, int taille)
2      {
3          int i, mini, j, x;
4
5          for (i = 0; i < taille - 1; i++)
6          {
7              mini = i;
8              for (j = i + 1; j < taille; j++)
9                  if (t[j] < t[mini])
10                     mini = j;
11              x = t[i];
12              t[i] = t[mini];
13              t[mini] = x;
14          }
15      }

```

Exercice 12 GCD en Event-B

Soit l'algorithme GCD calculant le pgcd de deux nombres entiers positifs quelconques a et b.

```

begin
start:
  while  $y_1 \neq y_2$  do
    loop:
      if  $y_1 < y_2$  then
        inloop1:
           $y_2 := y_2 - y_1$ ;
        endif1:
      else
        inloop2:
           $y_1 := y_1 - y_2$ ;
        endif2:
      fi
    endif
  end-while
endloop:
 $g := y_1$ ;
end

```

Question 12.1 Traduire l'algorithme en un module TLA^+ qui précise les annotations et qui précise les invariants pour la correction partielle et l'absence d'erreurs à m'exécution.

Question 12.2 Construire une machine Event-B qui liste les actions de l'algorithme et qui donne l'invariant pour la correction partielle et l'absence d'erreurs à l'exécution.